

DEBRISMIND

Autonomous Orbital Debris Collision Avoidance Intelligence

Complete Project Documentation
FAR AWAY 2026 International Hackathon

Space & Aerospace

Agentic AI

Multi-Agent Systems

Theme	Space & Aerospace + Agentic & Autonomous Systems
Hackathon	FAR AWAY 2026 — India's Biggest International Hackathon
Project Type	Software — Multi-Agent AI + 3D Web Dashboard
Cost to Build	Rs. 0 — Rs. 40 (fully free stack available)
Data Sources	CelesTrak TLE API, Space-Track.org (both free)
Core Tech	Python, FastAPI, React, CesiumJS, sgp4, LLM API

This document is a complete zero-to-launch guide for DebrisMind — covering the space debris crisis, all technical concepts from scratch, the full system architecture, build pipeline, and implementation roadmap. No prior knowledge of satellites or orbital mechanics is assumed.

Table of Contents

Chapter 1 — The Space Debris Crisis

- 1.1 What is a satellite?
- 1.2 What is space debris?
- 1.3 The scale of the problem
- 1.4 Why this matters right now

Chapter 2 — Key Concepts: Everything From Scratch

- 2.1 Orbits explained simply
- 2.2 TLE — Two-Line Element sets
- 2.3 SGP4 — the physics engine
- 2.4 Conjunction & TCA
- 2.5 Delta-V — the fuel of space
- 2.6 Multi-agent systems

Chapter 3 — The Problem Statement

- 3.1 Current state of collision avoidance
- 3.2 The gap DebrisMind fills
- 3.3 Why existing tools fall short

Chapter 4 — DebrisMind — What We Are Building

- 4.1 System overview
- 4.2 The novel idea: orbital negotiation
- 4.3 What the system does end-to-end

Chapter 5 — System Architecture & Pipeline

- 5.1 Architecture diagram
- 5.2 Data pipeline
- 5.3 Conjunction detector
- 5.4 Multi-agent negotiation engine
- 5.5 3D web dashboard
- 5.6 Data flow summary

Chapter 6 — Data Sources

- 6.1 CelesTrak — free TLE data
- 6.2 Space-Track.org
- 6.3 What the data looks like
- 6.4 How to fetch it

Chapter 7 — Tech Stack

- 7.1 Backend
- 7.2 AI / LLM layer
- 7.3 Frontend
- 7.4 Free alternatives for every paid tool

Chapter 8 — Build Guide: Step by Step

- 8.1 Environment setup
- 8.2 Step 1: Data pipeline

- 8.3 Step 2: Conjunction detector
- 8.4 Step 3: Agent layer
- 8.5 Step 4: React + CesiumJS frontend
- 8.6 Step 5: Connecting it all

Chapter 9 — The Demo Scenario

- 9.1 What judges will see
- 9.2 The 3-minute walkthrough
- 9.3 Backup plan for offline demo

Chapter 10 — Cost Breakdown & Free Stack

- 10.1 Full cost table
- 10.2 100% free configuration

The Space Debris Crisis

Understanding the problem from absolute zero — no prior knowledge needed

1.1 What is a Satellite?

A satellite is any object that orbits (travels around) the Earth. There are two kinds: natural satellites (like the Moon) and artificial satellites — machines we launch into space. When people say 'satellite' in the context of this project, they mean artificial satellites.

Satellites are launched into orbit by rockets. Once in orbit, they keep moving in a curved path around Earth because of a balance between their speed (which tries to throw them away from Earth) and gravity (which tries to pull them back down). This balance lets them orbit indefinitely without any fuel as long as they stay above the atmosphere.

What do satellites do?

Use Case	How it helps people
GPS Navigation	Your phone knows where you are because GPS satellites beam their location constantly.
Weather Forecasting	Weather satellites take photos of clouds and storms from above.
Internet (Starlink)	SpaceX's Starlink constellation gives internet access to rural areas worldwide.
Military Surveillance	Governments use satellites to monitor borders and troop movements.
Scientific Research	NASA's Hubble telescope is a satellite that photographs distant galaxies.
TV Broadcasts	Dish TV works because satellites relay TV signals from studios to homes.

How many satellites are there?

As of 2024, there are roughly 9,000 active satellites in orbit. But the number is growing explosively — SpaceX alone plans to have 42,000 Starlink satellites in orbit. Amazon, OneWeb, and dozens of other companies are also launching thousands more. By 2030, experts expect over 100,000 satellites to be operational simultaneously.

Key Fact

SpaceX launches up to 60 Starlink satellites in a single rocket. As of 2024, there are already over 6,000 Starlink satellites in orbit — more than all other satellites combined.

1.2 What is Space Debris?

Space debris (also called space junk) is any man-made object in orbit that is no longer functional. Think of it like litter — but orbiting Earth at 28,000 km/h.

Type of Debris	What it is
----------------	------------

Dead satellites	Old satellites that ran out of fuel or broke down — still in orbit, completely uncontrollable
Rocket bodies	The upper stages of rockets used to launch satellites — they stay in orbit after the launch
Explosion fragments	When a satellite or rocket explodes (due to leftover fuel igniting), it creates thousands of fragments
Collision fragments	When two objects collide, they shatter into thousands more fragments.
Paint flecks & bolts	Tiny objects like paint chips and screws lost during spacewalks — still dangerous at orbital speeds

How fast is space debris moving?

Objects in Low Earth Orbit (LEO) travel at approximately 7.8 kilometres per second — that is 28,000 km/h. A 1 cm piece of debris at this speed hits with the same energy as a bowling ball thrown at 90 km/h. A 10 cm piece can completely destroy a satellite. There is no way to dodge an untracked piece of debris.

Why size doesn't matter much in space

A 1 cm aluminium ball at orbital velocity carries enough kinetic energy to penetrate the walls of the International Space Station. At 28,000 km/h, a marble becomes a cannonball. This is why even small untracked objects are an existential threat to satellites.

1.3 The Scale of the Problem

Category	Count (approx. 2024)
Objects larger than 10 cm (tracked)	27,000+
Objects 1-10 cm (partially tracked)	500,000+
Objects smaller than 1 cm (untracked)	100,000,000+
Active satellites	~9,000
Dead satellites (debris)	~3,000
Rocket bodies in orbit	~2,000

1.4 Why This Matters Right Now

The Kessler Syndrome, proposed by NASA scientist Donald Kessler in 1978, describes a catastrophic chain reaction: one collision creates debris, that debris causes more collisions, which create more debris, until certain orbital shells become completely unusable — forever. No more GPS. No more weather satellites. No more internet from space.

In 2009, the first accidental satellite collision in history happened: Iridium-33 (an active US communications satellite) and Kosmos-2251 (a dead Russian military satellite) collided at 11.7 km/s, creating over 2,300 tracked fragments — which are still in orbit today. In May 2024, a severe G5 geomagnetic storm disrupted GPS globally — a preview of what infrastructure loss looks like. The difference with a debris cascade is it would be permanent.

The core insight

With 42,000+ Starlink satellites alone planned, plus thousands from other companies, the probability of conjunction events (near-collisions) will increase by orders of magnitude. The current manual, slow, satellite-by-satellite approach to collision avoidance WILL NOT SCALE. An autonomous, multi-satellite coordination system is not optional — it is existentially necessary.

Key Concepts: Everything From Scratch

Every technical term used in this project — explained simply

2.1 Orbits Explained Simply

Imagine throwing a ball horizontally. It curves downward due to gravity and hits the ground. Now throw it faster. It travels further before hitting the ground. Throw it fast enough (7.9 km/s), and the curve of its fall exactly matches the curve of the Earth — it never hits the ground. It is now in orbit.

Orbits are described by their altitude (height above Earth) and shape. The closer to Earth, the faster you must travel to stay in orbit, and the shorter your orbital period (time for one full revolution).

Orbit Type	Abbreviation	Altitude	Period	Used for
Low Earth Orbit	LEO	200 – 2,000 km	90–120 min	Starlink, ISS, spy sats
Medium Earth Orbit	MEO	2,000 – 35,786 km	2–24 hrs	GPS satellites
Geostationary Orbit	GEO	35,786 km	24 hrs (stationary)	TV, weather sats
Sun-Synchronous	SSO	~600 km	~97 min	Earth observation

Why DebrisMind focuses on LEO

LEO is the most crowded orbital shell — this is where Starlink, OneWeb, the ISS, and most debris live. The combination of high density and high relative speeds (up to 15 km/s during head-on approaches) makes LEO the most dangerous environment for collision events.

2.2 TLE — Two-Line Element Sets

A TLE (Two-Line Element set) is the standard format used worldwide to describe where a satellite is in its orbit and how it is moving. It is called 'two-line' because the data is packed into exactly two lines of text, 69 characters each. This format was invented by NORAD in the 1960s and is still used universally today.

What a TLE looks like

```
ISS (ZARYA)
1 25544U 98067A 24001.50000000 .00001234 00000-0 12345-4 0 9991
2 25544 51.6400 225.4800 0001234 67.8900 292.0200 15.49813200123456
```

What it means	Simple analogy
Human-readable name	Your name on your ID card
Unique ID (e.g. 25544 for ISS)	Your Aadhar/passport number
The date/time this data was valid	Timestamp of a GPS reading
Tilt of orbit relative to equator (degrees)	Angle of a road on a slope

Right Ascension of Ascending Node — where the orbit crosses equator going north	Longitude of the orbit's crossing point
How elliptical (egg-shaped) the orbit is; 0 = perfect circle	How oval vs circular the path is
How many revolutions per day	Laps per day on a racetrack
Where in the orbit the satellite is right now	Your position on a race track
How much atmosphere is slowing the satellite	Air resistance factor

Important

TLE data goes stale quickly — LEO satellites shift enough in 2-3 days that old TLEs give wrong positions. DebrisMind fetches fresh TLEs every few hours from CelesTrak, a free public service that updates constantly from NORAD tracking data.

2.3 SGP4 — The Physics Engine

SGP4 (Simplified General Perturbations 4) is a mathematical model — a set of equations — that takes a TLE and calculates where a satellite will be at any future time. It accounts for the fact that orbits are not perfect: the Earth is not a perfect sphere (it bulges at the equator), the atmosphere has trace amounts of drag even at 400 km altitude, and the Sun and Moon have tiny gravitational effects.

Think of SGP4 as the 'simulator' that runs time forward. You give it a TLE and say 'where will this satellite be in 6 hours?' and it gives you an X, Y, Z position in 3D space and velocity (how fast it is moving in each direction). This is not an approximation — it is the same model used by NORAD, NASA, and every space agency on Earth.

SGP4 in practice

The Python library 'sgp4' (pip install sgp4) implements the full SGP4 model. It can compute a satellite's position and velocity for any moment in time in under 1 millisecond on a standard laptop. For 200 satellites over a 7-day window (sampled every 60 seconds), the full computation takes under 5 seconds.

2.4 Conjunction & TCA

A conjunction is when two objects in orbit come dangerously close to each other. 'Dangerously close' is defined as coming within a threshold distance — typically 5 km for large satellites and 1 km for critical assets like the ISS.

TCA stands for Time of Closest Approach — the exact moment when the two objects are at their minimum distance from each other during a conjunction. This is the critical moment: if a maneuver is needed, it must be completed before TCA.

Conjunction	Event where two orbiting objects pass dangerously close
TCA	Time of Closest Approach — the moment of minimum separation
Miss Distance	The closest distance between two objects during a conjunction (in km or meters)

Probability of Collision (Pc)	Statistical probability that two objects will actually collide, accounting for tracking uncertainty
Conjunction Data Message (CDM)	Official notification sent when a conjunction is detected
Screening volume	The box of space around a satellite within which objects are monitored

2.5 Delta-V — The Currency of Space

Delta-V (written as delta-v, or Δv) literally means 'change in velocity.' In space, every maneuver — speeding up, slowing down, changing orbit — costs delta-V. Satellites carry a finite amount of propellant, and when it runs out, the satellite is dead. Delta-V is measured in meters per second (m/s).

A typical collision avoidance maneuver costs 0.1 to 1.0 m/s of delta-V. A Starlink satellite carries enough fuel for roughly 100–200 m/s of total delta-V over its lifetime. Each avoidance maneuver therefore consumes a meaningful fraction of the satellite's fuel budget. This is why DebrisMind's multi-agent negotiation is valuable — by choosing which satellite maneuvers, the system minimises total fuel expenditure across the constellation.

2.6 Multi-Agent Systems

A multi-agent system (MAS) is a network of independent software programs (agents) that each perceive their environment, make decisions, and take actions — and can communicate with each other. Think of it like a group of robots that each have their own brain but can talk to each other and coordinate their actions.

In DebrisMind, each satellite is represented by a software agent. The agent knows its satellite's orbital state (position, velocity, fuel level, mission priority) and can propose maneuver options. When two agents detect that their satellites are on a collision course, they negotiate: who moves, when, how much, and in which direction — minimising total fuel cost while guaranteeing safety.

The Problem Statement

What is broken today and why DebrisMind is needed

3.1 Current State of Collision Avoidance

Today, collision avoidance works like this: the US Space Force's 18th Space Control Squadron continuously tracks all objects larger than 10 cm. When their algorithms detect a conjunction, they send a Conjunction Data Message (CDM) to the satellite operator — usually 2 to 7 days before the TCA. The operator's team then manually analyses the CDM, decides whether to maneuver, calculates a safe burn, commands the satellite to execute the burn, and waits for confirmation.

This process has three fatal flaws that will cause it to break down completely as constellation sizes grow from hundreds to tens of thousands of satellites:

Problem	Description	Impact
Human bottleneck	A human team must manually analyse every CDM. At 6,000 conjunctions a day, that's 6,000 CDMs a day.	Stops satellites from operating at scale or being updated in real time
Each satellite acts alone	Satellite A doesn't know Satellite B is also planning to maneuver. They might collide.	Wasted fuel, possible collision
Days of lead time needed	Current maneuvers are planned 2-7 days in advance. Short-notice conjunctions (from tracked debris) have no time to plan.	No response to us (from tracked debris) have no time to plan.
No fuel optimisation	Each operator optimises for their own satellite's fuel. Nobody considers a fuel-optimal constellation — or across constellations.	Massive amounts of fuel wasted
No cooperation between operators	SpaceX and OneWeb cannot easily coordinate their satellites. Coordination is a huge challenge.	Coordination gaps at the international level

3.2 The Gap DebrisMind Fills

DebrisMind introduces a multi-agent coordination layer that sits between raw orbital data and the operator's command console. Instead of each satellite reacting independently, DebrisMind's agents communicate, negotiate, and produce a globally optimal maneuver plan — autonomously, in minutes rather than days.

What's missing today	What DebrisMind provides
Human analysts read CDMs one by one	AI agents process thousands of conjunctions simultaneously in seconds
Each satellite plans maneuvers alone	Agents negotiate to find the globally fuel-optimal solution
Maneuver decisions take days	Agent negotiation produces a plan in under 2 minutes
No cross-constellation coordination	Agent protocol works regardless of satellite owner
Outputs: raw data tables	Outputs: plain-English operator brief + visualised 3D maneuver plan

The novel contribution

The core innovation of DebrisMind is the 'orbital lane negotiation protocol' — treating conjunction avoidance as a multi-agent game-theoretic negotiation problem, rather than a single-satellite optimisation problem. No existing public tool or academic paper implements this specific approach. This is not a student project — it is a research-level idea wrapped in a buildable demo.

DebrisMind — What We Are Building

The complete system from input to output

4.1 System Overview

DebrisMind is a web-based platform with three layers: a Python backend that handles data ingestion and physics computation; an AI agent layer that handles negotiation and decision-making; and a 3D React frontend dashboard that visualises orbits, conjunctions, and maneuvers in real time.

Layer	Technology	Responsibility
Data Layer	Python + Celestrak API	Fetch TLE data, propagate orbits using SGP4, find conjunctions
Agent Layer	FastAPI + LLM API (Groq/Gemini)	Run multi-agent negotiation, compute optimal maneuvers, generate reports
Presentation Layer	React + CesiumJS + WebSockets	3D globe with live orbits, conjunction alerts, negotiation log, maneuver controls

4.2 The Novel Idea: Orbital Lane Negotiation

The key innovation is treating the avoidance problem as a negotiation between agents, not a solo calculation. Here is how it works conceptually:

- Step 1:** Two satellites are detected to be on a collision course. Miss distance: 1.2 km. TCA: 31 hours from now.
- Step 2:** Agent A (representing Sat-A) calculates three maneuver options it could perform — each with a different fuel cost and resulting miss distance.
- Step 3:** Agent B (representing Sat-B) does the same — three options on its side.
- Step 4:** Both agents submit their proposals to a Mediator Agent. Each proposal includes: proposed delta-V, resulting miss distance, fuel cost, and a natural-language justification ('My satellite is on a critical imaging pass, proposing minimum burn').
- Step 5:** The Mediator Agent evaluates all combinations — which pair of proposals produces safe miss distance at minimum total fuel cost — and selects the winner.
- Step 6:** The winning plan is formatted as an operator brief and displayed on the dashboard. Maneuver is shown in 3D on the globe.

4.3 What the System Does End-to-End

Input: A live TLE feed for 200+ Starlink satellites and known debris objects. Output: For every detected conjunction — a complete, human-readable operator brief with the recommended maneuver, fuel cost, new miss distance, and 3D visualisation on a globe.

Example output brief

CONJUNCTION ALERT — HIGH RISK Objects: STARLINK-1234 and COSMOS 1408 DEB TCA: 2026-06-09

14:32 UTC (31 hrs 14 min from now) Current miss distance: 1.2 km — BELOW SAFE THRESHOLD

Recommended action: STARLINK-1234 execute prograde burn Delta-V required: 0.31 m/s | Fuel cost: 8% of reserve New miss distance after maneuver: 47.3 km — SAFE Reason: COSMOS 1408 DEB is uncontrolled (debris) — only STARLINK can maneuver.

System Architecture & Pipeline

Every component explained in detail — what it does, how it connects, what data flows through it

5.1 Architecture Overview

The system is a classic three-tier web application, with an unusual middle layer: the multi-agent AI engine. Data flows from space (via TLE APIs) through physics computation, through AI negotiation, and out to the operator's browser.

External Sources	CelesTrak API Space-Track.org Historical TLE archive
Data Layer (Python)	TLE Fetcher → SGP4 Propagator → Conjunction Detector
Agent Layer (FastAPI)	Agent A + Agent B → Mediator Agent → Maneuver Planner → Brief G
Presentation Layer (React)	CesiumJS Globe + Conjunction Panel + Negotiation Log + Alert Feed
Operator	Views dashboard in browser – one-click to approve maneuver

5.2 Data Pipeline

The data pipeline is the foundation. It runs on a schedule (every 30–60 minutes) and refreshes the orbital state of every tracked object.

Step	Name	What happens	Input	Output
1	TLE Fetch	HTTP GET to CelesTrak for Starlink TLE group	URL request	Raw TLE text file
2	TLE Parse	Split into individual satellite records; extract name and TLE text	Raw TLE text	List of TLE objects
3	SGP4 Propagate	For each satellite, compute position+velocity every 50s for 7 days	Raw TLE text	Position arrays (ECI coords)
4	Conjunction Scan	For every pair, find minimum distance over 7-day position arrays	Position arrays	Conjunction list (pairs + TCA + miss dist)
5	Risk Filter	Keep only conjunctions with miss distance < 5 km	Full conjunction list	High-risk conjunction list
6	Alert Queue	Push each high-risk conjunction to the agent system	High-risk list	Agent task queue

5.3 Conjunction Detector

The conjunction detector is the most computationally intensive component. For N satellites, there are $N*(N-1)/2$ pairs to check — for 200 satellites, that is 19,900 pairs. For each pair, we compute the minimum distance over a 10,080-point timeline (7 days × 24 hrs × 60 min). Using numpy vectorisation, this runs in 3–8 seconds on a standard laptop.

The algorithm: propagate both satellites to the same set of timestamps using SGP4. Compute the Euclidean distance between their ECI (Earth-Centered Inertial) positions at each timestamp. Find the minimum. If the minimum is below the threshold, record the pair, the TCA timestamp, and the miss distance.

ECI coordinates explained

ECI (Earth-Centered Inertial) is a 3D coordinate system with the origin at Earth's center, where the X axis points toward a fixed star, not rotating with Earth. It is the natural coordinate system for orbital mechanics because satellites follow straight-line physics in this frame — no pseudo-forces from Earth's rotation.

5.4 Multi-Agent Negotiation Engine

For each conjunction in the alert queue, the agent system spawns a negotiation session. The session involves three agents: Agent A (for Satellite A), Agent B (for Satellite B), and the Mediator.

Agent	Input it receives	What it does	Output
Agent A	Sat-A orbital state, fuel budget, conjunction data	Generates 3 maneuver proposals (low/medium/high delta-V) with justification text	JSON list of 3 proposals with delta-V vectors
Agent B	Same for Sat-B	Same as Agent A	JSON list of 3 proposals
Mediator	All 6 proposals + conjunction data	Evaluates 9 combinations (3x3), checks each for safety, picks optimal maneuver	Final recommended action in plain English

How the LLM is used

Each agent is implemented as a Python function that calls an LLM API (Groq, Gemini, or OpenAI). The LLM receives a structured prompt containing the satellite's state as JSON and is instructed to return a JSON array of maneuver proposals. The LLM's role is reasoning: deciding which maneuver direction makes physical sense, weighing mission priority, and writing the justification text. The actual orbital mechanics (computing the new trajectory after each proposed burn) is done by the SGP4 library — not the LLM.

5.5 3D Web Dashboard

The frontend is a React single-page application using CesiumJS for the 3D globe. It connects to the FastAPI backend via WebSockets for real-time updates.

Panel	What it shows
3D Globe (CesiumJS)	Real Starlink orbits rendered as coloured lines on a photorealistic Earth. Conjunction points highlighted.
Conjunction Alert List	Live feed of detected conjunctions, sorted by risk level. Each shows: satellite names, TCA, miss distance.
Negotiation Log	Real-time transcript of agent messages as negotiation happens. Shows Agent A's proposals and Agent B's responses.
Operator Brief Panel	The final recommended action in plain English. Shows before/after miss distance, delta-V, and justification.
Constellation Stats	Total satellites tracked, conjunctions detected in last 24 hrs, maneuvers executed, total fuel consumed.

5.6 Data Flow Summary

From	Data transmitted	To
CelesTrak	TLE data (text)	TLE Parser
TLE Parser	Satellite objects	SGP4 Engine
SGP4 Engine	Position arrays (ECI)	Conjunction Detector
Conjunction Detector	High-risk pairs + TCA	Agent System
Agent A + B	Maneuver proposals (JSON)	Mediator Agent
Mediator Agent	Winning plan + brief	FastAPI WebSocket
FastAPI WebSocket	JSON events	React Frontend
React Frontend	3D visualisation + brief	Operator

Data Sources

Where every piece of data comes from — all free

6.1 CelesTrak — Primary TLE Source

CelesTrak (celestrak.org) is run by Dr. T.S. Kelso and is the most widely used free source of TLE data for researchers and developers. It mirrors data from NORAD/US Space Force and provides clean, pre-formatted TLE feeds for different object categories.

Data needed	URL
Starlink satellites	https://celestrak.org/SOCRATES/query.php OR https://celestrak.org/satcat/tle.php?GROUP=starlink
Space debris (all)	https://celestrak.org/SOCRATES/query.php
ISS	https://celestrak.org/satcat/tle.php?CATNR=25544
Active satellites	https://celestrak.org/satcat/tle.php?GROUP=active
Recent launches	https://celestrak.org/satcat/tle.php?GROUP=last-30-days

How to fetch TLE data in Python (3 lines)

```
import requests
r = requests.get('https://celestrak.org/SOCRATES/query.php?GROUP=starlink&FORMAT=tle')
tle_text = r.text # Contains all Starlink TLEs as plain text
```

6.2 Space-Track.org — Official Source

Space-Track.org is the official US Space Force catalog — the most complete and authoritative source. It requires a free account registration (takes 2 minutes) and provides a REST API. For a hackathon, CelesTrak is sufficient. Space-Track is useful if you want to show the full debris catalog (27,000+ objects).

6.3 What the Raw TLE Data Looks Like

A sample from the Starlink TLE feed (3 satellites shown):

```
STARLINK-1007
1 44713U 19074A 24001.50000000 .00001000 00000-0 99999-4 0 9994
2 44713 53.0500 165.2100 0001234 100.1200 259.8800 15.06375700 12345

STARLINK-1008
1 44714U 19074B 24001.50000000 .00001100 00000-0 10000-3 0 9991
2 44714 53.0500 165.2500 0001250 101.2000 258.7800 15.06380100 12346

STARLINK-1009
1 44715U 19074C 24001.50000000 .00001050 00000-0 10500-3 0 9992
2 44715 53.0500 165.3000 0001210 99.8900 260.1200 15.06372200 12347
```

6.4 How to Fetch and Parse in Python

```
import requests
from sgp4.api import Satrec, jday

# Fetch all Starlink TLEs
url = 'https://celestrak.org/SOCRATES/query.php?GROUP=starlink&FORMAT=tle'
lines = requests.get(url).text.strip().split('\n')

# Parse into satellite objects (every 3 lines = 1 satellite)
satellites = []
for i in range(0, len(lines)-2, 3):
    name = lines[i].strip()
    sat = Satrec.twoline2rv(lines[i+1], lines[i+2])
    satellites.append({'name': name, 'sat': sat})

# Propagate one satellite to a specific time
from datetime import datetime, timezone
dt = datetime(2026, 6, 9, 14, 0, 0, tzinfo=timezone.utc)
jd, fr = jday(dt.year, dt.month, dt.day, dt.hour, dt.minute, dt.second)
e, r, v = satellites[0]['sat'].sgp4(jd, fr)
# r = [x, y, z] position in km (ECI), v = velocity in km/s
```

Tech Stack

Every tool, library, and service used — with free alternatives for everything

7.1 Complete Tech Stack

Component	Tool / Library	Why this one	Free alternative
Orbital mechanics	sgp4 (Python)	Official SGP4 implementation, precise	Non-stdlib, MIT license, IS free
HTTP data fetch	requests (Python)	Standard library for HTTP calls	Built-in urllib
Numerical compute	numpy (Python)	Vectorised distance calculations	One-hundred-thousand point-pairs in seconds
API backend	FastAPI (Python)	Modern async Python API, WebSockets	Slack (simple, but no docs)
LLM (agents)	Groq API (Llama 3.1 70B)	Free tier: 14,400 req/day. Fast. Streaming	Slack (simple, but no docs)
3D globe	CesiumJS	Open source, photorealistic Earth	Three.js + NASA texture (full offline)
Frontend framework	React 18	Industry standard, huge ecosystem	Plain HTML + CSS (simple)
Real-time comms	WebSockets (FastAPI + browser)	Pushes negotiation updates to browser	WebRTC (every 2s)
TLE data source	CelesTrak API	Free, no auth, updated hourly, 25,000+ objects	Spacecraft.org (free signup)
Data validation	Pydantic (Python)	Ensures agent JSON outputs are well-formed	JSON processing
Package manager	pip + npm	Standard for Python and Node.js	

7.2 LLM API Comparison — Free Options

API	Free tier	Model quality	Speed	Best for
Groq (Llama 3.1 70B)	14,400 req/day free	Excellent — 70B model	Very fast (LPU hardware)	Primary choice
Google Gemini Flash	1,500 req/day free	Very good	Fast	Backup choice
Ollama (local)	Unlimited (runs on laptop)	Good (8B model)	Medium	Offline demo safety
OpenAI GPT-4o mini	No free tier	Excellent	Fast	If you have credits
Anthropic Claude Haiku	No free tier	Excellent	Fast	If you have credits

Recommended free configuration

Use Groq (free tier) as primary LLM. Set up Ollama with Llama 3.1 8B as a local fallback. Before the demo, cache 3 pre-computed negotiation scenarios. This gives you: zero API cost, fast responses, and a demo that works with no internet.

Build Guide: Step by Step

Complete implementation roadmap from a blank folder to a working demo

8.1 Environment Setup

```
# Create project structure
mkdir debrisMind && cd debrisMind
mkdir backend frontend

# Backend setup
cd backend
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install sgp4 numpy fastapi uvicorn requests pydantic websockets groq

# Frontend setup (in new terminal)
cd frontend
npm create vite@latest . -- --template react
npm install
npm install cesium resium axios
```

8.2 Step 1 — Data Pipeline (tle_pipeline.py)

File: backend/tle_pipeline.py. This fetches TLEs and propagates orbits.

```
import requests, numpy as np
from sgp4.api import Satrec, jday
from datetime import datetime, timedelta, timezone

def fetch_tles(group='starlink'):
    url = f'https://celestrak.org/SOCRATES/query.php?GROUP={group}&FORMAT=tle'
    lines = requests.get(url).text.strip().split('\n')
    sats = []
    for i in range(0, len(lines)-2, 3):
        name = lines[i].strip()
        sat = Satrec.twoline2rv(lines[i+1].strip(), lines[i+2].strip())
        sats.append({'name': name, 'sat': sat})
    return sats

def propagate(sats, hours=24, step_min=5):
    times = [datetime.now(timezone.utc) + timedelta(minutes=i)]
    for i in range(0, hours*60, step_min):
        results = {}
        for s in sats:
            positions = []
            for t in times:
                jd, fr = jday(t.year, t.month, t.day, t.hour, t.minute, t.second)
                e, r, v = s['sat'].sgp4(jd, fr)
                if e == 0: positions.append(r) # e=0 means success
            results[s['name']] = np.array(positions)
    return results, times
```

8.3 Step 2 — Conjunction Detector (conjunction.py)

File: backend/conjunction.py. Finds all close approaches.

```
import numpy as np
from itertools import combinations

def find_conjunctions(propagated, times, threshold_km=5.0):
    conjunctions = []
    names = list(propagated.keys())
    for nameA, nameB in combinations(names, 2):
        posA = propagated[nameA]
        posB = propagated[nameB]
        if len(posA) != len(posB): continue
        # Vectorised distance computation (numpy – very fast)
        diffs = posA - posB
        distances = np.linalg.norm(diffs, axis=1)
        min_idx = np.argmin(distances)
        min_dist = distances[min_idx]
        if min_dist < threshold_km:
            conjunctions.append({
                'satA': nameA, 'satB': nameB,
                'miss_km': round(float(min_dist), 3),
                'tca': times[min_idx].isoformat()
            })
    return sorted(conjunctions, key=lambda x: x['miss_km'])
```

8.4 Step 3 — Agent Layer (agents.py)

File: backend/agents.py. This is the core innovation. Each agent calls the LLM to generate maneuver proposals. The mediator picks the best combination.

```
import json
from groq import Groq # pip install groq

client = Groq(api_key='YOUR_FREE_GROQ_API_KEY')

def run_satellite_agent(sat_name, orbital_state, fuel_pct, priority, conjunction):
    prompt = f'''You are the autonomous agent for satellite {sat_name}.
    Orbital state: {json.dumps(orbital_state)}
    Fuel remaining: {fuel_pct}% | Mission priority: {priority}
    Conjunction: miss distance {conjunction['miss_km']} km at TCA {conjunction['tca']}
    Propose exactly 3 maneuver options. Return ONLY valid JSON array:
    [{{'delta_v_ms': float, 'direction': str, 'new_miss_km': float,
    'fuel_cost_pct': float, 'justification': str}}, ...]'''

    resp = client.chat.completions.create(
        model='llama-3.1-70b-versatile',
        messages=[{'role': 'user', 'content': prompt}],
        temperature=0.3
    )
    return json.loads(resp.choices[0].message.content)

def mediate(proposals_a, proposals_b, threshold=10.0):
    best, best_cost = None, float('inf')
    for pa in proposals_a:
        for pb in proposals_b:
            combined_miss = pa['new_miss_km'] + pb['new_miss_km']
            if combined_miss >= threshold:
                total_fuel = pa['fuel_cost_pct'] + pb['fuel_cost_pct']
                if total_fuel < best_cost:
                    best_cost = total_fuel
                    best = (pa, pb)
    return best
```

8.5 Step 4 — FastAPI Server (main.py)

```

from fastapi import FastAPI, WebSocket
from tle_pipeline import fetch_tles, propagate
from conjunction import find_conjunctions
from agents import run_satellite_agent, mediate

app = FastAPI()

@app.get('/api/conjunctions')
async def get_conjunctions():
    sats = fetch_tles('starlink')[:50] # limit to 50 for demo speed
    propagated, times = propagate(sats, hours=48, step_min=5)
    return find_conjunctions(propagated, times, threshold_km=5.0)

@app.websocket('/ws/negotiate')
async def negotiate(ws: WebSocket, satA: str, satB: str):
    await ws.accept()
    # ... spawn agents, stream negotiation updates to browser
    await ws.send_json({'type': 'agent_proposal', 'agent': 'A', ...})
    await ws.send_json({'type': 'mediator_decision', ...})

```

8.6 Step 5 — React + CesiumJS Frontend

Key components to build in React: a Cesium globe component that renders satellite orbits, a conjunction alert panel that polls `/api/conjunctions` every 30 seconds, a WebSocket hook that connects to `/ws/negotiate` and streams the negotiation live, and an operator brief card that displays the Mediator's output.

```

// Key CesiumJS snippet – render a satellite orbit as a polyline
import { Viewer, PolylineGraphics, Entity } from 'resium';
import { Cartesian3, Color } from 'cesium';

// orbitPoints = array of [lat, lon, alt] from your API
<Viewer full>
{orbitPoints.map((orbit, i) => (
<Entity key={i}>
<PolylineGraphics
positions={Cartesian3.fromDegreesArrayHeights(orbit.flat())}
width={1.5}
material={isConjunction ? Color.RED : Color.CYAN}
/>
</Entity>
)}}
</Viewer>

```

The Demo Scenario

Exactly what judges will see and how to make it unforgettable

9.1 The 3-Minute Walkthrough

Time	What happens on screen
0:00	Open dashboard. Globe loads with ~50 Starlink satellites orbiting in real-time as cyan lines. Judges can see them act
0:20	Click 'Scan for Conjunctions.' Two satellites light up in red. Alert panel shows: STARLINK-1234 and COSMOS 1408 D
0:40	Click 'Initiate Negotiation.' Negotiation log panel activates. Live stream: Agent A proposes 3 maneuvers. Agent B note
1:20	Mediator Agent selects Agent A's Option 2 — 0.31 m/s prograde burn. New miss distance: 47.3 km. Fuel cost: 8%.
1:40	Operator Brief appears: plain English summary of the situation and recommended action. Green 'Approve' button.
2:00	Click Approve. Globe animates STARLINK-1234's orbit shifting. Red lines turn green. Miss distance readout changes
2:30	Show constellation stats: 50 satellites monitored, 3 conjunctions detected in last 48 hrs, 2 maneuvers negotiated, esti

9.2 Backup Plan — Offline Demo

Always do this for hackathon demos

Before the event: run your full pipeline for 3 real conjunction scenarios. Save the outputs as JSON files. Build a 'demo mode' button in your frontend that loads these cached files instead of calling the live API. Your demo will work perfectly even with no internet, no API key, and no laptop power issues. Takes 1 hour to implement and saves the entire presentation.

9.3 Anticipating Judge Questions

Judge asks	Your answer
'Is this real data?'	Yes — we fetch live TLEs from CelesTrak, the same source used by researchers w
'Is the collision real?'	The conjunction is real in that it is computed from real orbital data. We either find a r
'Why is the AI needed? Can't you just do the math?'	The math tells you THAT there is a risk. The AI agents decide WHO maneuvers, WH
'Is this actually autonomous?'	Yes — from data fetch to conjunction detection to maneuver negotiation, no human
'Could this scale to 42,000 Starlink satellites?'	The conjunction detector and agent architecture are both parallelisable. At full scale

Cost Breakdown & Free Stack

Exactly what this costs to build and demo

10.1 Full Cost Table

Component	Tool	Cost	Notes
TLE data	CelesTrak	Free	No account, no key, unlimited requests
Orbital mechanics	sgp4 Python library	Free	Open source, MIT license
Numerical computing	numpy	Free	Open source
API backend	FastAPI	Free	Open source
LLM — primary	Groq API (free tier)	Free	14,400 req/day — more than enough
LLM — backup	Ollama local	Free	Runs on your laptop, no internet needed
3D Globe	CesiumJS + free Ion token	Free	Free Ion account takes 2 minutes to create
Frontend	React + Vite	Free	Open source
Hosting for demo	Run locally on laptop	Free	No server needed for hackathon demo
Domain / SSL	Not needed	Free	localhost is fine for demo
TOTAL		Rs. 0	Completely free if using Groq free tier
Worst case	If using OpenAI GPT-4o mini	Rs. 40	50 API calls for demo = pennies

10.2 Team Split Recommendation

Role	Who	Responsibilities	Time estimate
Backend / Physics	1 person (Python-strong)	TLE pipeline, SGP4 propagation, conjunction detector, FastAPI endpoints	1-2 weeks
AI / Agents	1 person (LLM experience)	Agent prompts, mediator logic, JSON parsing, brief generation, Websocket events	1-2 weeks
Frontend / Viz	1 person (React experience)	CesiumJS globe, conjunction panel, negotiation log, alert feed, orbit rendering	1-2 weeks

Final checklist before submitting

☐ GitHub repo created with clean README ☐ Celestrak fetch working and returning real TLEs ☐ SGP4 propagating at least 50 satellites ☐ Conjunction detector finding close approaches ☐ At least 1 agent negotiation working end-to-end ☐ CesiumJS globe rendering orbits in browser ☐ Demo scenario cached as JSON for offline backup ☐ 2-5 minute demo video recorded ☐ Presentation deck: problem, solution, demo screenshots, tech stack, novel contribution

DebrisMind — FAR AWAY 2026 | Space & Aerospace + Agentic & Autonomous Systems
All data from Celestrak (celestrak.org) | Physics: sgp4 library | 3D: CesiumJS | AI: Groq free tier | Cost: Rs. 0